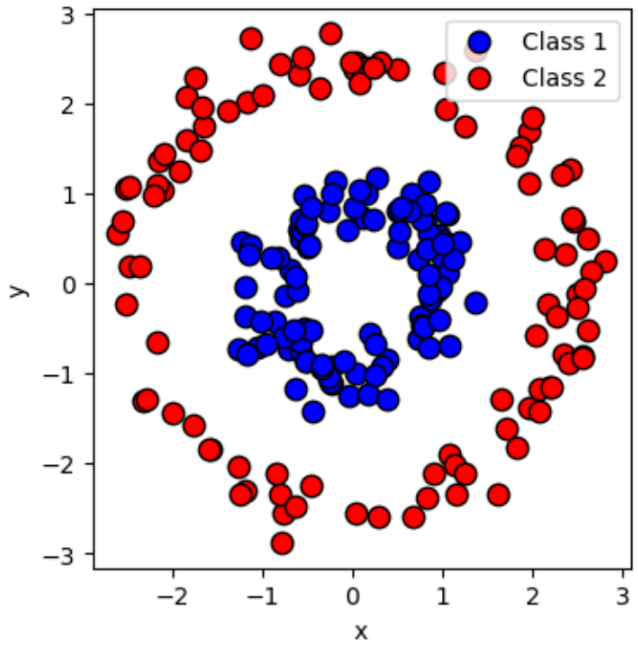
Neural Networks III

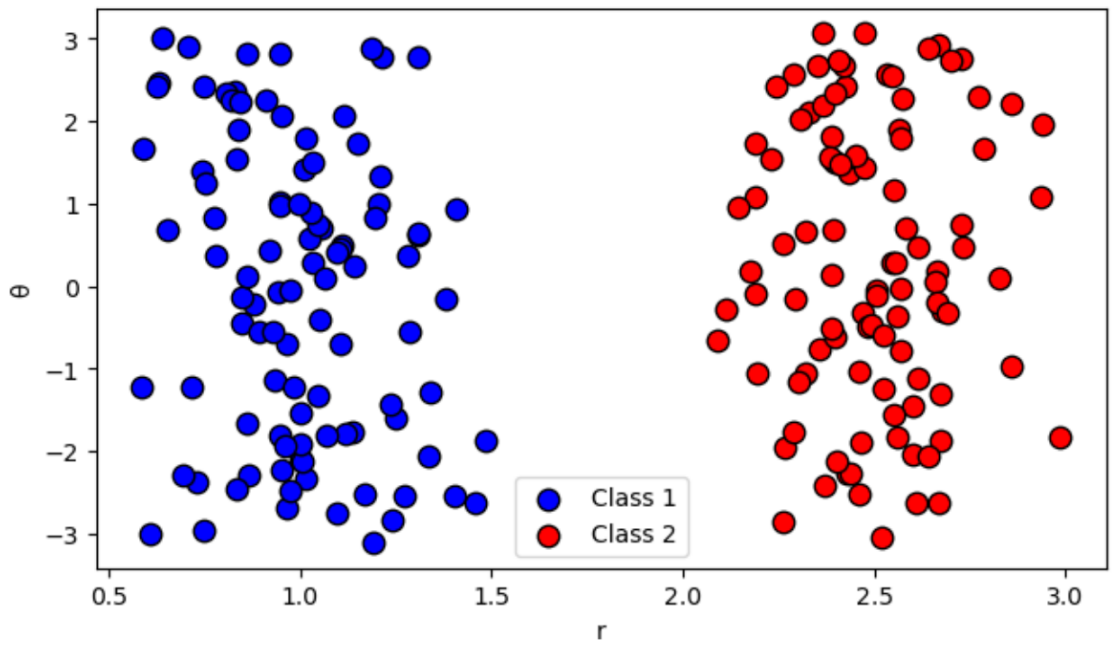
Introduction

- You have now learned about the fundamentals of neural networks.
- The goal of this lecture it to elevate your understanding of neural networks.

The core idea of neural networks - transform data into a new representation.

- Neural networks belong to the field of research called representation learning.
- Very useful in cases where the original representation is difficult to work with.
    - Images, text, graphs, etc.





The core idea of neural networks - transform data into a new representation

- Show video
- Visualizing Neural Networks with the Grand Tour

From scalars and sum to vectors and matrix multiplication

- Our derivation of Backpropagation two lectures back looked at scalar derivatives.
- Very little linear algebra and vector calculus.
- Simple operation, but leads to complicated computations
    - A lot of indices and sums.
- We will now continue the representation perspective to derive a much cleaner version of Backpropagation.
    - Bonus: also aligns much more closely to how the code should look!

A new perspective of neural networks - modules of computation

- Draw example.

Forward pass with linear algebra

- Before we looked at the pre-activation of one neuron: $z_j^l = (\mathbf{w}_j^l)^T \mathbf{a}^{l-1}$
- Note: augmented space!
- Now we look at the pre-activations of the whole layer: $\mathbf{z}^l =$

- Reminder: $\mathbf{a}^{l-1} =$

- And : $\mathbf{W}^l =$

Forward pass with linear algebra

- Usually want to process a batch of samples.
- Can also be done efficiently!
- Let a batch of inputs be represented as $\mathbf{A}^{l-1} = \begin{bmatrix} a_{1,1}^{l-1} & a_{2,1}^{l-1} & \cdots & a_{N,1}^{l-1} \\ a_{1,2}^{l-1} & a_{2,2}^{l-1} & \cdots & a_{N,2}^{l-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1,k_{l-1}}^{l-1} & a_{2,k_{l-1}}^{l-1} & \cdots & a_{N,k_{l-1}}^{l-1} \\ 1 & 1 & \cdots & 1 \end{bmatrix}$

- Then:

Backward pass with linear algebra and vector calculus

- Previously, for the output layer: $\frac{\partial}{\partial \mathbf{w}_j^L} E(i) = \frac{\partial}{\partial \mathbf{w}_j^L} z_j^L(i) \frac{\partial}{\partial z_j^L(i)} E(i)$
- Want: $\frac{\partial J}{\partial \mathbf{W}^l}$
- Difficult, ends up with a vector by matrix derivate.
- Start simpler: $\frac{\partial}{\partial \mathbf{w}_j^L} \mathbf{z}^L \frac{\partial}{\partial \mathbf{z}^L} \mathbf{e}^T \mathbf{e} \frac{1}{2}$
- Where we assume one-hot encoded labels and $\mathbf{e} = (\mathbf{a}^L - \mathbf{y})$

Finding δ for output layer

- We have dealt with the following term before $\frac{\partial}{\partial \mathbf{z}^L}\mathbf{e}^T\mathbf{e}\frac{1}{2} = \mathbf{e}\frac{\partial}{\partial \mathbf{z}^L}(\mathbf{a}^L - \mathbf{y})$
- $\mathbf{y}$ does not depend on $\mathbf{z}^L$, and we keep the derivative of $\mathbf{a}^L = f(\mathbf{z}^L)$ general.
- We have a vector by vector derivative $\Rightarrow$ Jacobian: $\frac{\partial \mathbf{a}^L}{\partial \mathbf{z}^L} =$

Finding δ for output layer

- Can be compatctly represented as $\frac{\partial}{\partial \mathbf{z}^L}\mathbf{e}^T\mathbf{e}^{\frac{1}{2}} =$

- $\odot$ is the Hadamard product or elementwise multiplication.

Backward pass with linear algebra and vector calculus

- Now, we turn to: $\frac{\partial}{\partial \mathbf{w}_j^L} \mathbf{z}^L$
- Vector by vector derivatve $\Rightarrow$ Jacobian matrix:

Derivative of one element in Jacobian

- Reminder: $z_1^L = w_{1,1} * a_1^{L-1} + w_{1,2} * a_2^{L-1} + \cdots w_{1,k_{L-1}} * a_{k_{L-1}}^{L-1} + 1 * w_{1,0}$
- Therefore: $\frac{\partial}{\partial w_{1,1}} z_1^L =$

- Back to Jacobian:

Putting it all together

- Combing both derivatives give:

Putting it all together

- Take a step back. Derivative of loss with respect to neuron 1 gave non-zero elements in column 1.
- If we repeat process of derivative with resepct to neuron j, coulmn j will be non-zero.
- We can get all derivatives with one matrix operation:
- $\frac{\partial J}{\partial \mathbf{W}^L} =$

What have we gained?

- A lot of work, but clean result with simple rule.
- To find gradients of loss with respect to weights in layer l, look at δ from current layer and output from previous layer.

A final touch on neural networks

- Many nice results for linear classifiers.
    - Optimal in terms of e.g. maximum likelihood.
    - Guarantueed unique solution for SVM.
- Hard to obtain similar results for neural networks due to their complexity.
- However, work is ongoing!

Connecting SVMs and neural networks

- The big question in neural network research:
  - Why do they generalize so well?
  - Seems to defy traditional statistics.
- One of many directions to answer this question:
  - Generalization due to implicit bias in gradient descent algorithm.
- Interesting work by Soundry et al (2018)

Connecting SVMs and neural networks

- Setting:
  - Neural network with one layer.
  - Binary classification.
  - Separable classes (similar results have been shown for non-separable classes).
  - Also some assumptions on loss function.

$$\mathbf{w}(t+1) = \mathbf{w}(t) - \eta \nabla \mathcal{L}(\mathbf{w}(t)) = \mathbf{w}(t) - \eta \sum_{n=1}^{N} \ell' \left(\mathbf{w}(t)^{\top} \mathbf{x}_n \right) \mathbf{x}_n. \tag{2}$$

Connecting SVMs and neural networks

- What happens when we let the number of iterations tend towards infinity?
- Remarkably, we obtain the weights of the hard margin SVM!
- They also show how the KKT conditions "naturally" fall out of this analysis.

Theorem 3 *For any dataset which is linearly separable (Assumption 1), any β-smooth decreasing loss function (Assumption 2) with an exponential tail (Assumption 3), any stepsize $\eta < 2\beta^{-1}\sigma_{\max}^{-2}(\mathbf{X})$ and any starting point $\mathbf{w}(0)$, the gradient descent iterates (as in eq. 2) will behave as:*

$$\mathbf{w}(t) = \hat{\mathbf{w}} \log t + \boldsymbol{\rho}(t) , \tag{3}$$

where $\hat{\mathbf{w}}$ is the L_2 max margin vector (the solution to the hard margin SVM):

$$\hat{\mathbf{w}} = \underset{\mathbf{w} \in \mathbb{R}^d}{\operatorname{argmin}} \|\mathbf{w}\|^2 \text{ s.t. } \mathbf{w}^\top \mathbf{x}_n \geq 1, \tag{4}$$

and the residual grows at most as $\|\boldsymbol{\rho}(t)\| = O(\log\log(t))$, and so

$$\lim_{t \to \infty} \frac{\mathbf{w}(t)}{\|\mathbf{w}(t)\|} = \frac{\hat{\mathbf{w}}}{\|\hat{\mathbf{w}}\|}.$$

Furthermore, for almost all data sets (all except measure zero), the residual $\rho(t)$ is bounded.

Wrapping up neural networks

- Neural networks are powerful and versatile tools.
- You have now learned the fundamentals. A lot more to be said.
    - However, this foundation often applies also for bigger and more modern neural networks.
- This course is your chance to code a neural network from scratch, take that opportunity!